

To Dew: Product Spec

2026-09-21 · @Someone

Overview

To Dew is a native macOS app that shows one list: what you intend to do today. At 6:00 AM the day turns over, and a short morning review decides what happens to yesterday's leftovers. The app lives in the Dock, is written by Claude, and ships as open source on GitHub.

Target system: macOS 26 (Tahoe), developed and tested on 26.4.1 (25E253). Minimum deployment target: macOS 26.0.

Principles

- **Today first.** The main window shows today and nothing else. Planning, backlog and history sit one step away.
- **Fast capture.** Adding a task takes one keystroke and a line of text.
- **Forgiving.** Nothing is ever lost at the 6 AM refresh. Every task ends up kept, scheduled, parked or archived.
- **Local and private.** All data stays on the Mac. No account, no network calls, no analytics.
- **Design comes last.** Logic and layout are built so a custom visual design can be applied without touching behavior.

Decisions so far

Topic	Decision
Name	To Dew
6 AM refresh	Morning review asks what to do with each unfinished task
Builder	Claude writes the code; spec is written for that workflow

Topic	Decision
Distribution	Open source on GitHub under the MIT license, no paid Apple Developer account
Visual style	Decided at the end; build must support a highly designed result
Task detail	Title, optional note, optional due time, priority flag
Beyond today	Recurring tasks, schedule for later, Someday backlog, history view
Extras	Desktop widget, reminders, Dock badge, streaks
Streak rule	A day counts when at least one task is completed
Windows	One main window only, plus an optional small floating icon above all windows
Sound	Silent by default
Evening nudge	On, at 9:00 PM
Language	English only at 1.0
1.0 scope	Only the features in this spec; none of the parked ideas
Not in scope	iCloud sync, iPhone app, menu bar quick-add, App Store release

The day model

A "day" in this app runs from 6:00 AM to 5:59 AM local time, not midnight to midnight. A task finished at 1:30 AM Tuesday counts toward Monday. This one rule drives the list, history, streaks and the widget.

- **Logical date.** Every timestamp maps to a logical date: subtract the rollover offset (default 6 hours), then take the calendar date. One function, `DayClock.logicalDate(for:)`, owns this. No other code computes dates.
- **Rollover is a state check, not a timer.** The app stores `lastOpenedLogicalDate`. Whenever the current logical date is later than the stored one, a rollover is due. A timer at 6:00 AM is only a convenience for when the app is open.

- **When the check runs.** On launch, on wake from sleep, when the window becomes active, on system clock or time zone change, and when the 6:00 AM timer fires.
- **Mac asleep or app closed at 6:00 AM.** Nothing is missed. The check runs at the next launch or wake and the morning review appears then.
- **Several days missed.** One review covers everything. Leftovers from all missed days are shown together, grouped by their original day. Recurring tasks generate only for today, not for each skipped day.
- **Time zones.** Days are stored as calendar dates (year-month-day) with no time zone. Travel changes when 6:00 AM happens but never rewrites history.
- **Clock moved backward.** If the logical date goes backward, do nothing. Never un-archive or duplicate a day.
- **Setting.** Rollover time is adjustable in Settings from 12:00 AM to 12:00 PM in 30-minute steps. Default 6:00 AM.

flowchart TD

```

    A[Launch, wake, focus,<br/>clock change, 6 AM timer] --> B{Logical date
    later<br/>than last opened?}
    B -- No --> C[Show today as is]
    B -- Yes --> D[Close out past days:<br/>archive completed tasks]
    D --> E[Generate today's recurring<br/>and scheduled tasks]
    E --> F{Any unfinished<br/>leftovers?}
    F -- Yes --> G[Morning review]
    F -- No --> H[Fresh today list]
    G --> H
  
```

The rollover runs as a single database transaction, so a crash midway can never leave a half-finished day.

Core features

The main window is a single Today list with a header, a progress indicator and an always-ready entry field.

Today window

- Header shows the logical date ("Monday, September 21"), a progress indicator (3 of 7 done) and the current streak.
- Tasks appear in manual order. Drag to reorder. Flagged tasks can optionally pin to the top (Settings).

- Completed tasks stay visible, dimmed, and sink below open tasks. A toggle hides them.
- Carried-over tasks show a small marker with how many days they have rolled ("2d").
- Empty state in the morning invites the first task. Empty state after finishing everything celebrates the day.
- Window size and position persist. Minimum size 360 by 480 points so it works as a narrow side panel.
- Sidebar or toolbar navigation to four other views: Upcoming, Someday, Recurring, History. The app has exactly one main window and views switch inside it. Settings is the only other window.

Floating icon (optional)

A small always-on-top element gives a glance at the day without opening the window. It is marked "maybe": build it in milestone 6, live with it for a week, then keep or cut it.

- About 44 points across. Floats above all other windows, on every Space and over full-screen apps.
- Shows today's progress as a ring with the open-task count. Becomes a check mark when everything is done.
- Click shows or hides the main window. Right-click lists the first five open tasks with click-to-complete, the same as the Dock menu.
- Drag it anywhere. It snaps to screen edges and remembers its position per display.
- Fades to low opacity when idle and never takes keyboard focus from the app in use.
- Off by default. Toggled in Settings or the View menu.
- Built as a non-activating `NSPanel` at floating window level. It is a panel, so the one-window rule still holds.

Task fields

Field	Type	Notes
Title	Text, required	Single line, up to 200 characters
Note	Text, optional	Multi-line plain text; links are clickable
Due time	Time, optional	A time on the task's day; fires a notification
Flag	Boolean	Marks a priority task

Field	Type	Notes
Day	Date or none	The logical date it belongs to; none means Someday
Status	Open, done, dropped	Dropped tasks are kept in history, not deleted
Carry count	Integer	How many times it has rolled over

Quick entry

- The entry field sits at the bottom of the list. Press Return to add and keep typing the next task.
- Light natural-language parsing on entry: "Call dentist 3pm" sets a due time, "!" at the end sets the flag, "tomorrow" or "fri" schedules it. Parsed tokens are highlighted and can be dismissed with Escape.
- Pasting multiple lines creates one task per line.

Keyboard

Shortcut	Action
Cmd N	Focus the entry field
Up / Down	Move selection
Space or Cmd Return	Toggle done
Return	Edit title inline
Cmd I	Open detail (note, due time)
Cmd Shift F	Toggle flag
Cmd Option Up / Down	Reorder
Cmd T	Move to tomorrow
Cmd Shift S	Move to Someday
Delete	Drop task (with Cmd Z undo)
Cmd 1 to 5	Switch view: Today, Upcoming, Someday, Recurring, History

Full undo and redo for every task action, through the standard Edit menu.

Morning review

The morning review is a short, skippable screen that appears the first time the app is opened after a rollover with unfinished tasks. It is the signature moment of the app and should take under 30 seconds.

Flow

1. **Yesterday recap.** One line: "Yesterday you finished 5 of 7." The streak updates here.
2. **Leftovers.** Each unfinished task appears as a card or row with four choices.
3. **Pull from Someday (optional).** A collapsed list of backlog tasks, each with an "Add to today" action.
4. **Start the day.** One button closes the review and lands on Today with the entry field focused.

Choices per leftover

Choice	Key	Result
Keep for today	K or Right arrow	Moves to today; carry count goes up by 1
Later	L	Pick tomorrow, a weekday or a date; leaves today
Someday	S	Moves to the backlog with no date
Drop	D or Left arrow	Marked dropped; visible in history, never deleted

Rules

- "Keep all" and "Drop all" buttons handle the whole list at once.
- Skipping or closing the review keeps every leftover for today. Nothing is lost by ignoring it.
- A task carried 3 or more times gets a gentle prompt: "This has rolled over 3 days. Schedule it, park it or drop it?" The threshold is a setting.
- No leftovers means no review. The app opens straight to a fresh Today list.
- Unfinished instances of recurring tasks are not shown. They close as "missed" and today's new instance takes their place.

- The review can be reopened any time that day from the View menu, until the first task is completed.
- A setting turns the review off. With it off, leftovers roll to today automatically.

Planning beyond today

Three secondary views hold everything that is not today: Upcoming for dated tasks, Someday for undated ones, and Recurring for templates.

Upcoming (schedule for later)

- Shows future tasks grouped by day: Tomorrow, then the next 7 days by weekday name, then by date.
- A scheduled task is hidden from Today until its day's rollover, then appears automatically.
- Tasks can be added directly to a future day, or dragged between days.
- Due time, note and flag can all be set ahead of time.

Someday (backlog)

- A single undated list with manual ordering.
- Actions: add to today, schedule for a date, drop.
- Shown inside the morning review as an optional pull list.
- Tasks older than 60 days get a subtle "stale" marker to encourage pruning. No automatic deletion.

Recurring tasks

- A recurring task is a template. At each rollover the app creates a normal task instance for today when the rule matches.
- Editing an instance changes only that day. Editing the template changes future instances.
- Templates can be paused without deleting them.

Rule	Example
Every day	Take vitamins

Rule	Example
Weekdays	Check inbox at 9
Chosen weekdays	Gym on Mon, Wed, Fri
Every N days	Water plants every 3 days
Monthly on a date	Pay rent on the 1st

- Monthly on the 29th to 31st falls back to the last day of shorter months.
- "Every N days" counts from the last generated instance, not the last completion.
- Each template has its own small history: a row of recent days marked done or missed.

History, streaks and stats

A day counts toward the streak when at least one task is completed during that logical day. History is a calendar of past days that can each be opened and read.

Streak rules

- Current streak is the run of consecutive successful days ending yesterday, plus today once today has a completion.
- Today never breaks the streak while it is still in progress. The streak only breaks at rollover.
- A day with zero tasks on the list is neutral. It neither extends nor breaks the streak. This keeps weekends and holidays from punishing the user.
- Longest streak is stored and shown beside the current one.
- Un-checking the only completed task of today removes today from the streak again.
- Streaks can be hidden entirely in Settings for users who dislike them.

History view

- Month calendar. Each day cell shows completion as a fill or ring (for example 5 of 7), so a month reads as a heat map.
- Selecting a day shows its read-only list: done, dropped, carried forward and missed recurring tasks.
- Search across all past tasks by title and note.
- A task in history can be duplicated into today with one action.

Stats

Stat	Definition
Current streak	Consecutive successful days
Longest streak	Best run on record
Completion rate	Tasks done divided by tasks on the list, over 7 and 30 days
Tasks completed	Totals for this week, this month, all time
Busiest weekday	Weekday with the highest average completions
Most carried task	The open task with the highest carry count

Data retention

History is kept forever by default. Settings offers export to JSON and Markdown, plus a "delete history older than" option.

Notifications, Dock badge and widget

All three are driven by the same local data and all can be turned off individually.

Notifications

Local notifications only, through UserNotifications. macOS delivers scheduled notifications even when the app is closed.

Notification	When	Default
Morning	At rollover time: "Your day is ready. 3 tasks carried over."	On
Due time	At a task's due time, with Done and Snooze 15 min actions	On
Evening nudge	At 9:00 PM (adjustable) if nothing is completed yet, to protect the streak	On
Streak milestone	At 7, 30, 100 and 365 days	On

- Permission is requested the first time the user sets a due time or enables a reminder, not at first launch.
- The morning notification is scheduled as a repeating daily trigger. Its text is refreshed whenever tasks change, since it cannot compute counts at delivery time.
- Clicking a notification opens the app to the relevant task or to the morning review.
- Respect Focus modes. No custom sounds by default.

Dock

- Badge shows the count of open tasks for today. Options: open count, flagged only, or off.
- Dock menu (right-click): New Task, the first five open tasks with click-to-complete, and Show Today.
- The Dock icon can optionally reflect the day state (clear, in progress, all done). This is a stretch goal, decided with the visual design.

Desktop widget

Size	Content
Small	Progress ring, done and total counts, current streak
Medium	Top 4 open tasks with checkboxes, plus progress
Large	Up to 10 tasks with checkboxes, progress and streak

- Built with WidgetKit. Checkboxes use App Intents so tasks complete from the desktop without opening the app.
- The widget timeline includes an entry at the next rollover time, so it flips to the new day even if the app is closed.
- The app reloads widget timelines after every data change.
- The widget reads the same store as the app through a shared App Group container.
- The widget must render well in the tinted and clear desktop styles, which strip color. It cannot rely on color alone.
- Signing risk: see Distribution. The widget is a separate build milestone so the core app never depends on it.

Design readiness

The visual style is chosen last, so the build must make restyling cheap: every color, font, size, shape and animation comes from one theme layer, and no view hard-codes appearance.

Requirements for the build

- **Design tokens.** A single `Theme` type holds colors, type styles, spacing, corner radii, shadows, materials and animation curves. Views read tokens from the SwiftUI environment. No literal colors, font sizes or paddings inside views.
- **Swappable themes.** `Theme` is a protocol or value with at least two implementations from day one (a plain default and a deliberately different test theme). This proves the token layer works before the real design arrives.

- **Component library.** Reusable pieces built once and used everywhere: TaskRow, Checkbox, ProgressRing, DayHeader, EntryField, ReviewCard, CalendarCell, StatTile, EmptyState, FloatingIcon. Each has an Xcode preview showing every state.
- **Custom checkbox.** The checkbox is a custom view with its own animation hook, not the system toggle. Completion is the most repeated interaction and deserves the most polish.
- **Motion hooks.** Named animation points exist even if they start simple: task complete, task add, task remove, reorder, day rollover transition, review card swipe, all-done celebration, streak increment.
- **Window chrome.** Use a hidden title bar with full-size content so the design can run edge to edge. Traffic lights stay in place.
- **Materials.** Background supports solid, system material and Liquid Glass options through the theme. Glass is applied only to floating chrome such as toolbars and the entry field, never to content rows.
- **Light, dark and accent.** Every token has light and dark values. The user's system accent color is respected unless the theme overrides it.
- **Typography.** Text uses semantic styles (title, body, caption) mapped in the theme, so a custom font can be swapped in later. Dynamic text size is supported.
- **Sound and haptics.** A `Feedback` service handles completion sounds and trackpad haptics. Sound is off by default; the app stays silent until the user turns it on in Settings.
- **App icon.** Built with Icon Composer as a layered icon so it supports the default, dark, clear and tinted macOS 26 appearances. A placeholder ships until the design phase.

Quality bar that does not wait for design

- Every state is designed for: empty, loading, one task, 50 tasks, very long titles, all done.
- Accessibility: full VoiceOver labels and actions, complete keyboard access, Reduce Motion and Increase Contrast honored, no information carried by color alone.
- 60 fps scrolling and animation with 500 tasks in a list.
- Text never truncates without a way to read it in full.

Design phase (final milestone)

Pick a direction, produce mockups of Today, Morning review, History and the widget, then implement it as a new `Theme` plus motion and icon work. No logic changes should be needed.

Technical architecture

A SwiftUI app with SwiftData storage, split into a logic package with no UI and a thin UI layer. The split lets Claude test the hard parts (dates, rollover, recurrence, streaks) from the command line.

Area	Choice
Language and UI	Swift 6, SwiftUI, AppKit only where SwiftUI falls short (Dock menu, window tuning)
Storage	SwiftData (SQLite) in the App Group container
Widget	WidgetKit plus App Intents
Notifications	UserNotifications, local only
Dependencies	None. Apple frameworks only
Tests	Swift Testing for the logic package; a few XCTest smoke tests
Tooling	Xcode 26; builds from the command line with <code>xcodebuild</code>

Project layout

- `DailyCore` (Swift package): `models`, `DayClock`, `RolloverEngine`, `RecurrenceEngine`, `StreakCalculator`, `QuickEntryParser`, import and export. No SwiftUI imports.
- `DailyUI` (Swift package): `Theme`, components, views.
- `App` target: app entry, windows, menus, Dock integration, notification scheduling, Settings.
- `Widget` target: widget views and App Intents, depending on `DailyCore`.

Rules for the code

- The clock is injected. `DayClock` takes a `now` provider so tests can simulate any date, sleep gap or time zone jump.
- Rollover is idempotent. Running it twice for the same day changes nothing.
- All writes go through one store actor so the app and widget never conflict.

- Wake detection uses `NSWorkspace.didWakeNotification` ; clock changes use `NSSystemClockDidChange` and `NSSystemTimeZoneDidChange` .
- Schema versions are declared from the first release, so later updates can migrate user data safely.
- Optional "Launch at login" through `SMAppService` .
- English only at 1.0. All user-facing text still goes through a String Catalog, so other languages can be added later without code changes.
- The floating icon is the one place that needs AppKit windowing: an `NSPanel` hosting a SwiftUI view.

Data model

Entity	Key fields
Task	id, title, note, day (date or none), dueTime, isFlagged, status, sortIndex, carryCount, createdAt, completedAt, originalDay, templateID
RecurringTemplate	id, title, note, dueTime, isFlagged, rule, isPaused, lastGeneratedDay, createdAt
DayRecord	day, totalCount, doneCount, droppedCount, isSuccessful, reviewedAt
AppState	lastOpenedLogicalDay, rolloverMinutes, longestStreak, schemaVersion

- `DayRecord` is written at rollover as a frozen summary. History and streaks read these rows, so they stay fast and stable even if old tasks are later pruned.
- `status` values: open, done, dropped, missed (recurring only).
- `day` is stored as a year-month-day value with no time zone.

Settings

Rollover time, morning review on or off, carry-count prompt threshold, flagged tasks pin to top, hide completed, badge mode, each notification type, evening nudge time (default 9:00 PM), completion sound (default off), floating icon (default off), show streaks, launch at login, theme, export and import, delete old history.

Distribution on GitHub

Without a paid Apple Developer account the app cannot be notarized, so the primary install path is building from source. A prebuilt download is offered as a convenience with clear Gatekeeper instructions.

Path	Who it suits	What they do
Build from source	Anyone with Xcode 26	Clone, open the project, pick their own free Personal Team for signing, press Run
Prebuilt zip from Releases	People without Xcode	Download, move to Applications, approve once under System Settings, Privacy and Security, Open Anyway

What the free route costs

- Downloaded builds are ad-hoc signed and not notarized. macOS blocks the first launch until the user approves it in System Settings.
- No automatic updates through a signed channel. The app can check the GitHub Releases feed and show a "new version available" link. This is the only network call and it is off by default.
- **Widget risk.** The widget shares data through an App Group, which depends on code signing identity. It should work when a user builds with their own free Personal Team. It may not work in the ad-hoc prebuilt zip. This is unverified and is the first technical spike in the milestones.
- Fallback if the spike fails: the prebuilt zip ships without the widget, and the README says the widget needs a source build.
- If a paid account is added later, only the release script changes. Nothing in the app needs rework.

Repository contents

- `README.md` with screenshots, install steps for both paths and the Gatekeeper walkthrough.
- `SPEC.md`, a Markdown copy of this document, so Claude and contributors build from the same source of truth.

- `CLAUDE.md` with build commands, test commands, architecture rules and the "no literal styling in views" rule.
- `LICENSE` : MIT.
- `CHANGELOG.md` and semantic version tags.
- GitHub Actions workflow: build and run `DailyCore` tests on every push; build the release zip on version tags. Needs a runner image with Xcode 26.
- Signing settings live in an `.xcconfig` file with a blank team ID, so contributors set their own without editing the project file.
- Bundle identifier and App Group ID are derived from one config value for the same reason.

Edge cases and acceptance tests

Each row below becomes an automated test in `DailyCore` , using the injected clock. The rollover logic is not done until all of them pass.

Scenario	Expected result
Task completed at 1:30 AM Tuesday	Counts toward Monday
App open across 6:00 AM	List rolls over live; review appears; no restart needed
Mac asleep from 11 PM to 8 AM	Rollover runs on wake; one review
App closed for 5 days	One review with leftovers grouped by original day; recurring tasks generated for today only
Rollover triggered twice in the same day	Second run changes nothing
Crash in the middle of rollover	On relaunch, the day is either fully old or fully new
Fly New York to Los Angeles	Past days unchanged; next rollover at 6:00 AM Pacific
Daylight saving change night	Exactly one rollover; no skipped or doubled day
System clock set back one day	No rollover, no duplicate records

Scenario	Expected result
User changes rollover time from 6 AM to 4 AM at 5 AM	Rollover happens immediately, once
Review skipped	All leftovers appear in today with carry count increased
Recurring daily task left undone	Yesterday's instance marked missed; one fresh instance today
Monthly task on the 31st in a 30-day month	Generated on the 30th
Task scheduled for a day that was skipped	Appears in today at the next rollover as a leftover
Only completed task of today is un-checked	Today leaves the streak
Day with zero tasks	Streak neither grows nor breaks
Task checked in the widget while the app is open	App list updates within 1 second
Widget visible at 6:00 AM with the app closed	Widget switches to the new day on its own
500 tasks in one day	Scrolling stays smooth; launch under 1 second
Title of 200 characters, or emoji, or right-to-left text	Displays fully; no clipping

Build milestones

Ten milestones, each ending in something that runs and can be checked. Logic comes before UI, and the visual design comes last, as requested.

#	Milestone	Done when
0	Spikes and scaffold	Repo, packages and CI exist. Widget plus App Group proven (or ruled out) under free signing. Empty app shows a Dock icon.

#	Milestone	Done when
1	Core logic	<code>DailyCore</code> models, <code>DayClock</code> , rollover, streaks. All acceptance tests pass from the command line.
2	Today list	Add, complete, edit, reorder, drop, undo. Data persists. Keyboard shortcuts work.
3	Rollover and morning review	Live rollover, wake handling, full review flow with keyboard.
4	Planning views	Upcoming, Someday, Recurring templates and generation, quick-entry parsing.
5	History and stats	Calendar heat map, day detail, search, stats, export and import.
6	Notifications and Dock	All four notification types, badge, Dock menu, floating icon, Settings window, launch at login.
7	Widget	Three sizes, interactive checkboxes, rollover-aware timeline.
8	Visual design	Direction chosen, mockups approved, final theme, motion, sounds, app icon.
9	Release	Accessibility pass, performance pass, README with screenshots, tagged 1.0 on GitHub.

Working with Claude on the build

- Milestone 1 suits Claude best: pure Swift with tests it can run and fix on its own.
- UI milestones need a human in the loop. Claude cannot see the running app unless given screenshots, so plan to run each build and report back.
- Keep `SPEC.md` and `CLAUDE.md` in the repo and ask Claude to update them when a decision changes.
- One milestone per branch and pull request keeps changes reviewable.

Recommendations and open questions

Version 1.0 ships exactly the features above and nothing more. Below are ideas parked for later, things left out on purpose, and the answers to the earlier open questions.

Parked for after 1.0

Feature	Why	Cost
Global quick-add hotkey	Capture a task from any app without switching windows. Works without a menu bar icon.	Small
Focus one task	A mode that shows only the current task, large, with the rest hidden. Fits a highly designed app.	Small
Daily intention line	One optional sentence at the top of the day, set in the morning review.	Small
Shortcuts and Siri actions	App Intents built for the widget can also expose "Add task" and "Show today" to Shortcuts nearly for free.	Small
Spotlight indexing	Find tasks from Spotlight.	Small
Plain-text export per day	A Markdown file per day for journaling or Obsidian users.	Small
End-of-day wrap-up	Optional evening screen: what got done, what moves to tomorrow. Mirrors the morning review.	Medium
Import from Apple Reminders	One-time import through EventKit for people switching over.	Medium

Leave out on purpose

- Projects, tags, subtasks and attachments. They turn a daily list into a task manager and make the design harder to keep clean.
- Sync and accounts. Revisit only if an iPhone version becomes a goal; SwiftData keeps that door open.
- Time tracking, Pomodoro timers and calendars. Separate products.
- AI features. Nothing here needs them, and they would break the local-only promise.

Answered questions

Question	Answer
App name	To Dew

Question	Answer
Float above other windows?	Not the window. Possibly a small floating icon; see Core features
More than one window?	No. One main window
Completion sound by default?	No. Silent until enabled
Evening nudge?	Yes, at 9:00 PM
License	MIT
Parked ideas in 1.0?	None
Languages	English only

Still open

- ☐ Bundle identifier prefix, usually `com.<your GitHub username>.todew`. Needed at milestone 0.
- ☐ Name styling: "To Dew", "to dew" or "ToDew" in the Dock and menu bar. Can wait for the design phase.
- ☐ Keep or cut the floating icon after trying it in milestone 6.